DAY4: MEMORIES & ARRAYS

Verilog Arrays:

In Verilog, an **array** allows you to group multiple elements of a data type together.Arrays can be declared for standard nets and variables, including reg, wire, integer, and time data types.

Vectors vs. Arrays It is important not to confuse a multi-bit vector with an array:

- A **vector** is a single element that is *n*-bits wide (e.g., a single 8-bit register).
- An **array** is a collection of multiple distinct elements, where each individual element can either be a 1-bit scalar or a multi-bit vector.

Syntax and Declaration When declaring an array of vectors, the bit-width (word size) is specified to the *left* of the variable name, and the array depth (number of elements) is specified to the *right*.

Examples of array declarations include:

- reg y1 [11:0]; declares a scalar array with a depth of 12, where each element is 1-bit wide.
- reg [7:0] mem [0:255]; declares an array containing 256 elements, where each element is an 8-bit vector. A one-dimensional array of reg variables like this is frequently used to model physical storage (like RAMs and ROMs) and is formally referred to as a **memory**.
- reg [7:0] y3 [0:1][0:3]; declares a multi-dimensional 2D array representing 2 rows and 4 columns, where each element is 8 bits wide.

Rules for Access and Assignment

- **Element Access:** To access a particular element in an array, you must specify an index for every defined dimension.
- **Bit-Level Access:** To access a specific individual bit within a memory word, you must use separate sets of brackets (e.g., mem[word_index][bit_index]).
- **No Bulk Assignments:** In standard Verilog, you cannot assign a value to an entire array at once (e.g., writing y1 = 0; or mem_a = mem_b will trigger a syntax error). To initialize or copy an array, you must either assign each index individually or iterate through the elements using a for loop.

```
y3[1][2] = 8'hdd;      // Assign 0xdd to rows=1 cols=2
y3[0][0] = 8'haa;      // Assign 0xaa to rows=0 cols=0
```

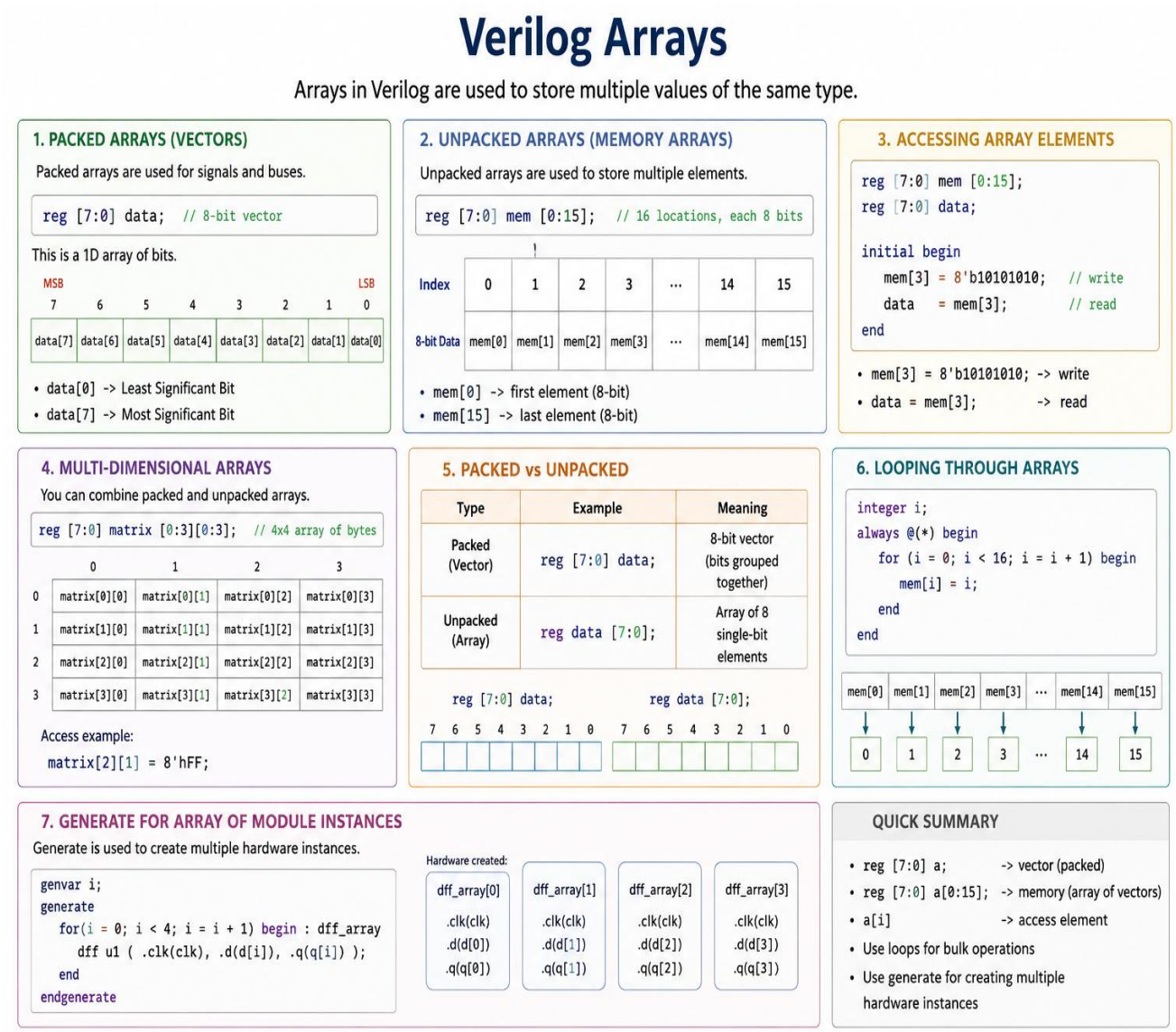
- **Port Connections:** When passing an array element to a module port, avoid using non-constant variable indices directly in the port map (e.g., .data_in(mem[i])). Synthesis tools frequently struggle with dynamic indexing in this context. It is much safer to assign the indexed value to a temporary wire or reg before passing it into the sub-module.

```
module des ();
  reg [7:0] mem1;
  reg [7:0] mem2 [0:3];
  reg [15:0] mem3 [0:3][0:1];

  initial begin
    int i;
    mem1 = 8'ha9;
    $display ("mem1 = 0x%0h", mem1);

    mem2[0] = 8'haa;
    mem2[1] = 8'hbb;
    mem2[2] = 8'hcc;
    mem2[3] = 8'hdd;
    for(i = 0; i < 4; i = i+1) begin
      $display("mem2[%0d] = 0x%0h", i, mem2[i]);
    end

    for(int i = 0; i < 4; i += 1) begin
      for(int j = 0; j < 2; j += 1) begin
        mem3[i][j] = i + j;
        $display("mem3[%0d][%0d] = 0x%0h", i, j, mem3[i][j]);
      end
    end
  end
endmodule
```



Verilog Memories:

In Verilog, **memories** are digital storage elements used to model hardware components like RAMs, ROMs, and FIFO (First-In-First-Out) queues. They are fundamentally modeled as **one-dimensional arrays of reg variables**.
Each element within this array represents a distinct storage "word," which can be accessed using a single array index.

Syntax and Declaration: It is crucial to distinguish between a single multi-bit vector and a memory array. When declaring a memory, the bit-width (word size) is defined to the *left* of the variable name, while the depth (number of entries) is defined to the *right*.

// Declares a memory array containing 256 words, where each word is 8 bits wide
reg [7:0] my_memory [0:255];

Accessing Memory

- **Word Access:** You read or write to a specific location by indexing the array, such as my_memory[address].
- **Bit-Level Access:** To access a specific bit within a specific memory word, you must use two separate sets of brackets, such as my_memory[word_index][bit_index].

Hardware Synthesis and Best Practices In real-world applications, memories are extensively used to build components like Network Packet Buffers in Ethernet switches and Wi-Fi routers.
When writing synthesizable RTL code for large memories, the best practice is to design a clocked always block that utilizes a single read/write address. This specific coding style is important because it allows the logic synthesis tool to efficiently infer dedicated **"Block RAM" (BRAM)** hardware on the chip, saving thousands of physical flip-flops and significant hardware resources.

Simulation and Initialization If you are simulating a memory (like a ROM) and need to load it with initial data, Verilog provides specific system tasks for this purpose. You can use **$readmemb** to initialize the memory from a file containing binary numbers, or **$readmemh** to initialize it from a file containing hexadecimal numbers.

Memory Example:

In this example, **register** is an array that has four locations with each having a width of 16-bits. The design module accepts an additional input signal which is called **addr** to access a particular index in the array.

```
module des (  input  clk,
                         input  rstn,
                         input [1:0]   addr,
                         input  wr,
                         input  sel,
                         input [15:0]  wdata,
                         output [15:0] rdata);

      reg [15:0] register [0:3];
      integer i;

      always @ (posedge clk) begin
            if (!rstn) begin
                  for (i = 0; i < 4; i = i+1) begin
                        register[i] <= 0;
                  end
```

```
            end else begin
                if (sel & wr)
                    register[addr] <= wdata;
                else
                    register[addr] <= register[addr];
            end
        end
        assign rdata = (sel & ~wr) ? register[addr] : 0;
endmodule
```

Register Vector:

In Verilog, a **register vector** is a single multi-bit storage element. This is distinct from a scalar which is only 1-bit wide and an array which is a collection of multiple distinct elements.
Declaration and Syntax Vectors are declared by specifying a bit-size range inside brackets strictly to the **left** of the variable name.
For example:
reg [15:0] my_register; //declares a single 16-bit wide register vector.

Hardware Realization:
For example, a 16-bit register vector will synthesize into a parallel bank of 16 flip-flops. You can then use control signals (like clock, reset, and write-enable) within a clocked procedural block to update the entire vector simultaneously during writes, or return its complete current value during reads.

Verilog Memories

In Verilog, memories are implemented using arrays of multi-bit registers.

1. MEMORY DECLARATION

General form:

```
reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];
```

Word Width — Number of Locations

Example:

```
reg [7:0] mem [0:15];   // 16 locations, 8-bit each
```

Memory Contents (8 bits)

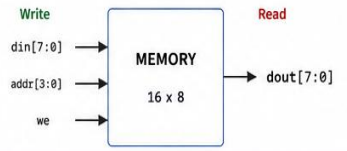
Address		7	6	5	4	3	2	1	0
0	mem[0]	0	1	0	1	1	0	1	0
1	mem[1]	1	0	0	1	0	1	0	1
2	mem[2]	1	1	1	0	0	0	1	1
3	mem[3]	0	0	1	0	1	1	0	1
...	...	...		...	...		...	...	
14	mem[14]	0	1	1	0	0	1	1	0
15	mem[15]	1	1	0	0	1	0	1	1

- Each location stores a word (here 8 bits).

2. READ / WRITE EXAMPLE (SYNCHRONOUS)

```
module mem_example(
    input clk,
    input we,              // Write Enable
    input [3:0] addr,      // Address (0 to 15)
    input [7:0] din,       // Data In
    output reg [7:0] dout  // Data Out
);
    reg [7:0] mem [0:15];  // 16 x 8 memory

    always @(posedge clk) begin
        if (we)
            mem[addr] <= din;     // Write
        else
            dout <= mem[addr];    // Read
    end
endmodule
```



3. MEMORY INITIALIZATION

Using initial block:

```
reg [7:0] mem [0:15];
integer i;
initial begin
   for (i = 0; i < 16; i = i + 1)
      mem[i] = i;      // mem[i] = 0,1,2,..,15
end
```

Using $readmemh (from file):

```
$readmemh("mem_init.hex", mem);
```

- File `mem_init.hex` contains hex values.

4. TYPES OF MEMORIES

Type	Description	Example
Single Port RAM	One address bus for read/write	`reg [7:0] mem [0:15];`
Dual Port RAM	Separate ports for read and write	Two address buses
ROM (Read Only)	Read only memory	Initialized memory

5. MULTI-DIMENSIONAL MEMORY

Example: 4 x 4 memory of 8-bit words

```
reg [7:0] mem2d [0:3][0:3];
```

Column

Row	0	1	2	3
0	mem2d[0][0]	mem2d[0][1]	mem2d[0][2]	mem2d[0][3]
1	mem2d[1][0]	mem2d[1][1]	mem2d[1][2]	mem2d[1][3]
2	mem2d[2][0]	mem2d[2][1]	mem2d[2][2]	mem2d[2][3]
3	mem2d[3][0]	mem2d[3][1]	mem2d[3][2]	mem2d[3][3]

6. COMMON OPERATIONS

Write Operation

```
mem[5] = 8'hA5;      // Write 0xA5 to address 5
```

Read Operation

```
data = mem[5];       // Read from address 5
```

Loop Through Memory

```
integer i;
 for (i = 0; i < 16; i = i + 1)
   $display("mem[%0d] = %h", i, mem[i]);
```

7. KEY POINTS

- Memories are arrays of vectors.
- `reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];`
- Used to model RAM, ROM, FIFOs, buffers, etc.
- Read/Write can be synchronous or asynchronous.
- Large memories are mapped to block RAM in FPGA.
- $readmemh / $readmemb are used for initialization.

ARRAY vs MEMORY

Array (General)

- Can be single-bit or multi-bit elements
- Used for small collections or registers
- Example: `reg flag [7:0];  // 8 one-bit elements`

VS

Memory (Special Type of Array)

- Each element is multi-bit (word)
- Used to store large amount of data
- Example: `reg [7:0] mem [0:15];  // 16 words, 8 bits each`

Important:
Every memory is an array, but every array is not necessarily a memory.

Verilog Parameter:

In Verilog, parameters are used to define constants that are evaluated at compilation or elaboration time.
To make your module definitions highly flexible, readable, and reusable.
They are most frequently used to define fundamental design characteristics such as data bus widths, array sizes, clock frequencies, or state encodings for finite state machines

Module Parameter:

In Verilog, **module parameters** are used to define constants that are evaluated at compilation or elaboration time.

Module parameters can be used to override parameter definitions within a module and this makes the module have a different set of parameters at compile time. A parameter can be modified with the defparam

Common Uses in Modules Parameters are most frequently utilized in module definitions to handle:

- **Sizing and Configuration:** Defining data bus widths, array depths, or system clock frequencies (e.g., parameter DATA_WIDTH = 8; or parameter NUM_REGS = 16;).
- **State Machine Encodings:** Defining readable named states for Finite State Machines (FSMs) (e.g., parameter s0 = 2'b00; parameter s5 = 2'b01;) .

Parameter Arithmetic Verilog allows you to perform mathematical operations when defining a module's parameters, provided that all the operands are other parameters or known constant numbers. Examples include:

- parameter ADDR_WIDTH = $clog2(NUM_REGS);
- parameter TOTAL_BUS_SIZE = DATA_WIDTH * 2;.

Because the compiler evaluates these arithmetic expressions entirely during elaboration time rather than runtime, the resulting value is simply a fixed constant.

Parameter Example:
The module **counter** has two parameters **N** and **DOWN** declared to have a default value of 2 and 0 respectively. **N** controls the number of bits in the output effectively controlling the width of the counter. By default it is a 2-bit counter. Parameter **DOWN** controls whether the counter should increment or decrement. By default, the counter will decrement because the parameter is set to 0.

2-bit up-counter

```
module counter
 #(      parameter N = 2,
         parameter DOWN = 0)

 ( input clk,
 input   rstn,
 input en,
       output reg [N-1:0] out);

 always @ (posedge clk) begin
 if (!rstn) begin
 out <= 0;
 end else begin
 if (en)
 if (DOWN)
 out <= out - 1;
```

```
 else
       out <= out + 1;
 else
 out <= out;
 end
 end
endmodule
```

Test Bench:

```
module design_top ( input clk,
 input rstn,
 input en,
 output [1:0] out);

 counter #(.N(2)) u0 (      .clk(clk),
 .rstn(rstn),
 .en(en));
endmodule
```

See that default parameters are used to implement the counter where **N** equals two making it a 2-bit counter and **DOWN** equals zero making it an up-counter. The output from counter is left unconnected at the top level.

4 Bit down counter:

```
module design_top ( input clk,
 input rstn,
 input en,
 output [3:0] out);

 counter #(.N(4), .DOWN(1))
              u1 (    .clk(clk),
 .rstn(rstn),
 .en(en));
endmodule
```

Overriding Module Parameters:

To reuse a generic module for different sizes or purposes, Verilog allows you to override its default parameter values. The formal syntax defines two distinct ways to achieve this:

1. **Instance Parameter Assignment:** You can pass new parameter values directly within the module instantiation statement using the # ( ) syntax (formally defined as <name_of_module> # (<expression>) <module_instance>) . As noted in our previous conversation, this looks like bus_master #(4, 5) b1();
2.
   Formal Syntax: This is formally defined as <name_of_module> <parameter_value_assignment>? <module_instance>, where the parameter override is structured as # (<expression> <, <expression>>*) .

3. **The defparam Statement:** You can explicitly override a specific parameter at the hierarchical level using the defparam keyword (formally defined as defparam <list_of_param_assignments>;)

   Formal Syntax: The syntax is explicitly defined as <parameter_override> ::= defparam <list_of_param_assignments>;, where each assignment targets a specific hierarchical identifier and assigns it a new <constant_expression> .

   Example:
   The first part instantiates the module called **design_ip** by the name **d0** where new parameters are passed in within #( ).
   The second part uses a Verilog construct called defparam to set the new parameter values.

```
module tb;
    // Module instantiation override
    design_ip #(BUS_WIDTH = 64, DATA_WIDTH = 128) d0 ( [port list]);

     // Use of defparam to override
     defparam d0.FIFO_DEPTH = 128;

endmodule
```

Local Parameter:

In Verilog, **local parameters** are used to define constants within a module using the **localparam** keyword.

The Key Difference:
localparam values cannot be modified or overridden from outside the module using a defparam statement or module instance parameter assignments. Attempting to override a local parameter externally will immediately throw an elaboration error during compilation. This strict "encapsulation" ensures that internal values remain constant throughout the module's lifetime, providing a way to protect critical design values from accidental external changes.

Common Applications and Best Practices A common beginner mistake is using the standard parameter keyword for internal constants, which exposes your internal logic to accidental overrides.

- **Finite State Machine (FSM) Encodings:** Local parameters are strongly recommended for defining internal FSM states (such as IDLE, READ, or WRITE). If an external user were able to accidentally override a state encoding using a standard parameter, the entire state machine logic would break. Using localparam safely "locks" these bit patterns inside the module.
- **Fixed Mathematical Values:** They should be used to protect internal mathematical constants that must remain constant regardless of the module's instantiation.
- **Encapsulation:** As a best practice, you should reserve parameter strictly for values you want to configure during instantiation (like bus or array widths), and use localparam for values internal to the module's logic. This makes your IP cores much more robust and safer for others to use.

Derived Constants and Bit-Selects Although local parameters cannot be overridden directly, you can assign them constant expressions that contain standard parameters. If the parent parameter is modified externally, the local parameter will automatically update its internal derived value (for example, evaluating localparam LENGTH = 2 + PARAM_LENGTH;). Additionally, Verilog allows you to perform bit-selects and part-selects on local parameters, provided they are not of the real data type.

```
module adder #(parameter WIDTH = 8, PARAM_LENGTH = 4) (
 input wire [WIDTH-1:0] a,
 input wire [WIDTH-1:0] b,
 output wire [WIDTH-1:0] sum
);
localparam LENGTH = 2 + PARAM_LENGTH;
assign sum = a + b;
endmodule




module tb;
 // Statements

 // Regular parameter PARAM_LENGTH can be changed even though it is
assigned to a localparam
 adder #(.WIDTH(16), .PARAM_LENGTH(5)) add_instance2 ( ... );
endmodule
```

Specify Parameter:

In Verilog, a **specify parameter** is a special type of constant declared using the **specparam** keyword, and it is primarily used to define timing characteristics and pin-to-pin delay values.

Purpose and Best Practices Instead of hardcoding numerical delay values directly into your path delay statements, it is highly recommended to define them first using specparam. This coding practice ensures that if the timing specifications of the circuit change, you only need to update the delay values in one centralized location rather than hunting through the code. Furthermore, specparam values are frequently used to store necessary delay data for non-simulation tools, such as external delay calculators, layout estimators, and synthesis tools.

Key Characteristics vs. Module Parameters :

- **Scope:** specify parameters are allowed to be declared either inside a specify block or directly within the main module body.
- **Assignments:** A specparam can be assigned the values of other specparams as well as standard parameters. However, standard module parameters cannot be assigned the value of a specparam.
- **Overriding Values:** Standard parameters are overridden via the defparam statement or module instance assignments (e.g., #()). Specify parameters **cannot** be overridden this way. Instead, specparam values are typically overridden and updated post-layout using the Standard Delay Format (SDF) during delay back-annotation.

Example:

```
// Use of specify block
specify
       specparam t_rise = 200, t_fall = 150;
       specparam clk_to_q = 70, d_to_q = 100;
endspecify
// Within main module
module my_block ( ... );
       specparam dhold = 2.0;
       specparam ddly = 1.5;
       parameter WIDTH = 32;
endmodule
```

VERILOG PARAMETERS

Parameters make Verilog modules configurable and reusable.

1. MODULE PARAMETERS

Defined using 'parameter' in the module header.

```
module adder #(
    parameter WIDTH = 8
)(
    input  [WIDTH-1:0] a,
    input  [WIDTH-1:0] b,
    output [WIDTH-1:0] sum
);
    assign sum = a + b;
endmodule
```

Default value:

WIDTH = 8

This is an 8-bit adder by default.

Why Use Parameters?

Without parameters:

```
module adder4(...);
module adder8(...);
module adder16(...);
...
```

→

With parameters:

One module for all sizes.

✅ Reusable & Scalable

2. OVERRIDING MODULE PARAMETERS

You can override parameter values during module instantiation.

Method 1: Named Parameter Override (Recommended)

```
adder #(
   .WIDTH(16)
) u1 (
   .a(a),
   .b(b),
   .sum(sum)
);
```

Now module becomes

Method 2: Positional Override (Less Preferred)

```
adder #(16) u1 (
   .a(a),
   .b(b),
   .sum(sum)
);
```

⚠ Less preferred because order matters. Named override is recommended.

EXAMPLE: PARAMETERIZED COUNTER

Module

```
module counter #(
    parameter N = 4
)(
    input clk,
    output reg [N-1:0] q
);
    always @(posedge clk)
        q <= q + 1;
endmodule
```

Instantiation

```
counter #(.N(8)) c1 (
   .clk(clk),
   .q(q)
);
```

Now:

q becomes 8-bit

3. LOCAL PARAMETERS (localparam)

localparam is like a parameter, but it CANNOT be overridden.

```
module example #(
   parameter WIDTH = 8
);
   localparam DEPTH = WIDTH * 2;   // Fixed internally

   localparam IDLE  = 2'b00;       // FSM states
   localparam START = 2'b01;
   localparam DONE  = 2'b10;
endmodule
```

Here:

- WIDTH → can be changed during instantiation
- DEPTH, IDLE, START, DONE → cannot be changed

Why Use localparam?

To protect internal constants and FSM states from accidental modification.

4. parameter vs localparam

Feature	parameter	localparam
Can Override?	Yes	No
Used For	User configuration (module customization)	Internal constants (derived values, FSM states)
Scope	External + Internal	Internal only
Example	parameter WIDTH = 8;	localparam DEPTH = WIDTH * 2;
Purpose	To make modules configurable	To keep internal values fixed

5. SPECIFY PARAMETERS (specparam)

Used inside specify blocks for timing specification.
Mostly used in gate-level modeling and ASIC timing simulation.

Example

```
module and_gate(
    input  a,
    input  b,
    output y
);
    and (y, a, b);

    specify
        specparam tpd = 5;      // delay value
        (a => y) = tpd;         // from a to y
        (b => y) = tpd;         // from b to y
    endspecify
endmodule
```

Meaning

When input 'a' changes, output 'y' changes after 5 time units. Same for input 'b'.

Usage

- ASIC timing
- Gate-level simulation
- SDF annotation (back-annotated delays)
- Rarely used in FPGA RTL

specparam can be used ONLY inside specify block and CANNOT be overridden.

6. COMPLETE COMPARISON

Type	Override Allowed?	Where Used	Purpose
parameter	Yes	Module header	Configurable values (width, depth, size, etc.)
localparam	No	Inside module	Internal constants, FSM states, derived values
specparam	No	Inside specify block	Timing delays for simulation

7. PRACTICAL USAGE EXAMPLES

Parameterized FIFO

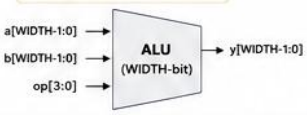
```
module fifo #(
   parameter WIDTH = 8,
   parameter DEPTH = 16
)(...);
```

Parameterized RAM

```
module ram #(
    parameter ADDR_WIDTH = 4,
    parameter DATA_WIDTH = 8
)(...);
```

Parameterized ALU

```
module alu #(
    parameter WIDTH = 16
)(...);
```



Parameterized Counter

```
module counter #(
    parameter N = 8
)(...);
```

8. parameter vs `define

parameter	`define (macro)
Module Specific	Global (everywhere)
Typed	Text substitution
Has scope	No scope
Can be overridden	No
Safer & better	Prone to errors

9. KEY TAKEAWAYS

- ✅ Use **parameter** to make modules configurable and reusable.
- ✅ Use **named** parameter override for clarity and safety.
- ✅ Use **localparam** for internal constants that should not change.
- ✅ Use **specparam** inside specify block for timing delays.
- ✅ Parameters are widely used in RTL designs like RAM, FIFO, counters, ALUs, UART, SPI, AXI, etc.

Verilog Arrays vs Memories

In Verilog, memories are a special type of array used to store multi-bit data words.

Arrays	Feature	Memories
An array is a general collection of elements of the same type. Elements can be 1-bit or multi-bit.	**Definition**	A memory is an array where each element is a multi-bit word used to store data.

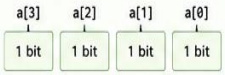
Example Declaration
`reg a [7:0];  // array of 8 single-bit elements`
`reg [3:0] b [0:3];  // array of 4 elements, each 4 bits`
	Declaration	Example Declaration
		`reg [7:0] mem [0:15];  // 16 locations, 8 bits each`

Elements can be 1-bit or multi-bit.	**Element Size**	Each element (word) is multi-bit.
Used for grouping signals, storing small collections of values, counters, flags, lookup tables, etc.	**Purpose / Usage**	Used for storage of large amounts of data such as RAM, ROM, FIFOs, buffers, caches, etc.
Accessed like normal variables or vectors. Usually not address-based storage.	**Access**	Accessed using an address (index). Behaves like a memory location.
Represents individual registers or wires. (Each element is a separate signal.)	**Hardware Meaning**	Represents addressable storage. (Like RAM or ROM block.)

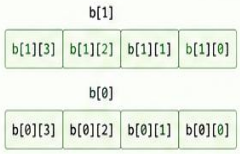
Examples
`reg a [3:0];  // 4 one-bit elements: a[0]..a[3]`
`reg [7:0] b [0:1];  // 2 elements, each 8 bits`
	Example	Example
		`reg [7:0] mem [0:7];  // 8 locations, 8 bits each`

Array (Examples & Visualization)

1. `reg a [3:0];`
   `// array of 4 single-bit elements`



2. `reg [3:0] b [0:1];`
   `// 2 elements, each 4 bits`



Memory (Example & Visualization)

`reg [7:0] mem [0:3];`
`// 4 locations, each 8 bits`

Address	Data (8 bits)							
mem[0]	7	6	5	4	3	2	1	0
mem[1]	7	6	5	4	3	2	1	0
mem[2]	7	6	5	4	3	2	1	0
mem[3]	7	6	5	4	3	2	1	0

Key Points

- Every memory is an array, but every array is not necessarily a memory.
- Use arrays for small collections of signals.
- Use memories for storing large amounts of multi-bit data.
- Memories are usually inferred as RAM/ROM in synthesis.
- Arrays may be used for holding control or status information.

Quick Comparison

Feature	Array	Memory
What it is	General collection of elements	Special type of array used for storage
Element width	Can be 1-bit or multi-bit	Always multi-bit (word)
Access method	Direct indexing, not necessarily address-based	Address-based access
Typical use	Signals, small data sets, control	RAM, ROM, FIFOs, buffers, large data storage
Example	`reg a[7:0];` `reg [3:0] b[0:3];`	`reg [7:0] mem[0:15];`
Hardware	Individual registers/wires	Addressable memory block (RAM/ROM)

Memory Read / Write Example

```
reg [7:0] mem [0:15];
reg [3:0] addr;
reg [7:0] din, dout;
reg we, clk;
always @(posedge clk) begin
    if (we)
        mem[addr] <= din;   // write
    else
        dout <= mem[addr];   // read
end
```